Projet Virtual Trader - Rapport de Projet

Auteurs : IF3A

• Elio

• Bilel

• Eliott

Sommaire

1. Introduction

2. Architecture de la Base de Données

    1. Schéma Entité-Association (Conceptuel)

    2. Schéma Relationnel de la Base de Données

3. Structure générale du projet

    1. Organisation des dossiers et fichiers

4. Présentation du projet réalisé : Virtual Trader

    1. Objectifs du projet et travail réalisé

    2. Gestion des utilisateurs et authentification

    3. Fonctionnalités principales du jeu de trading

    4. Simulation du marché et dynamique du jeu

    5. Fonctionnalités sociales et compétitives

    6. Ergonomie et qualité de l’application

    7. Bilan des fonctionnalités

5. Conclusion

Introduction

Ce rapport détaille la conception et la réalisation du projet “Virtual Trader”, une application web de simulation de trading d’actions. Développé par Elio, Bilel et Eliott. Ce projet vise à offrir une plateforme interactive et éducative permettant aux utilisateurs de s’initier aux mécanismes des marchés financiers sans encourir de risques réels en utilisant de l’argent fictif. L’objectif principal était de créer un environnement dynamique où les utilisateurs peuvent gérer un portefeuille virtuel, acheter et vendre des actions, suivre l’évolution des cours, percevoir des dividendes, et même contracter des emprunts. Une dimension sociale et compétitive a également été intégrée à travers un système de classement des joueurs et la possibilité de suivre d’autres utilisateurs. Ce document présente l’architecture de la base de données de l’application, la structure générale des fichiers du projet, ainsi qu’une description détaillée des fonctionnalités implémentées. Une attention particulière sera portée aux choix techniques effectués et à la manière dont ils répondent aux exigences du cahier des charges et aux critères d’évaluation. Nous aborderons également les aspects spécifiques tels que la simulation du temps et la gestion des comptes administrateurs, qui enrichissent l’expérience utilisateur et la robustesse de l’application. L’ambition de ce rapport est de fournir une vision claire du travail accompli, des défis relevés et des solutions apportées pour faire de Virtual Trader une application fonctionnelle, ergonomique et conforme aux attentes.

Architecture de la Base de Données

L’architecture de la base de données est un élément central du projet Virtual Trader. Elle a été conçue pour répondre aux besoins fonctionnels de l’application, notamment la gestion des utilisateurs, des actions, des transactions, des porte-feuilles, ainsi que le suivi des performances et des interactions sociales entre joueurs. Une attention particulière a été portée à la normalisation afin d’éviter la redondance des données et de garantir leur intégrité.

Schéma Entité-Association (Conceptuel)

Le schéma Entité-Association ci-dessous présente les principales entités du système et les relations qui les lient. Il offre une vue d’ensemble de la manière dont les données sont structurées au niveau conceptuel.

Entités principales :

1. UTILISATEUR

   • Attributs : id (PK), email, mot_de_passe, date_inscription, liquidites, derniere_connexion

   • Rôle : Représente un joueur ou un administrateur du système.

2. ACTION

• Attributs : id (PK), code, nom, description, prix_actuel, evolution_precedente, date_dividende, montant_dividende

• Rôle : Représente une action financière disponible à l'achat ou à la vente.

3. PRET

• Attributs : id_pret (PK), montant_emprunte, taux_interet_annuel, date_emprunt, date_echeance, montant_rembourse, statut_pret

• Rôle : Représente un emprunt effectué par un utilisateur.

Entités d'association (issues de relations N:N) :

4. POSSESSION_ACTION (Association entre UTILISATEUR et ACTION)

• Attributs : id (PK), utilisateur_id (FK vers UTILISATEUR), action_id (FK vers ACTION), quantite, prix_moyen_achat

• Cardinalités : Un UTILISATEUR peut posséder Plusieurs ACTIONS, Une ACTION peut être possédée par Plusieurs UTILISATEURS (N:N).

5. TRANSACTION

• Attributs : id (PK), utilisateur_id (FK vers UTILISATEUR), action_id (FK vers ACTION), type (achat/vente), quantite, prix_unitaire, montant_total, date_transaction

• Rôle : Historise chaque achat ou vente d'action par un utilisateur.

• Relation : Liée à UTILISATEUR (1:N, un utilisateur effectue N transactions) et à ACTION (1:N, une action peut être dans N transactions).

6. HISTORIQUE_PRIX

• Attributs : id (PK), action_id (FK vers ACTION), prix, date

• Rôle : Historise l'évolution du prix de chaque action.

• Relation : Liée à ACTION (1:N, une action a N historiques de prix).

7. HISTORIQUE_PORTEFEUILLE

• Attributs : id (PK), utilisateur_id (FK vers UTILISATEUR), valeur, date

• Rôle : Historise l'évolution de la valeur du portefeuille de chaque utilisateur.

• Relation : Liée à UTILISATEUR (1:N, un utilisateur a N historiques de portefeuille).

8. DIVIDENDE_RECU

• Attributs : utilisateur_id (FK vers UTILISATEUR, PK), date (PK), montant

• Rôle : Historise les dividendes reçus par un utilisateur pour une date donnée.

• Relation : Liée à UTILISATEUR (1:N).

9. SUIVI_JOUEUR (Association entre UTILISATEUR et UTILISATEUR)

• Attributs : id_suivi_joueur (PK), id_suiveur (FK vers UTILISATEUR), id_suivi (FK vers UTILISATEUR), date_debut_suivi

• Cardinalités : Un UTILISATEUR (suiveur) peut suivre Plusieurs UTILISATEURS (suivis), Un UTILISATEUR (suivi) peut être suivi par Plusieurs UTILISATEURS (suiveurs) (N:N).

10. HISTORIQUE_CLASSEMENT_JOUEUR

• Attributs : id_historique (PK), id_joueur (FK vers UTILISATEUR), date_snapshot, rang, valeur_portefeuille, performance_mensuelle

• Rôle : Historise le classement des joueurs à différentes dates.

• Relation : Liée à UTILISATEUR (1:N).

Relations principales (en dehors des tables de jointure explicites) :

• UTILISATEUR (1) — (0,N) POSSESSION_ACTION (Chaque possession concerne un utilisateur)

• ACTION (1) — (0,N) POSSESSION_ACTION (Chaque possession con- cerne une action)

• UTILISATEUR (1) — (0,N) TRANSACTION (Chaque transaction est effectuée par un utilisateur)

• ACTION (1) — (0,N) TRANSACTION (Chaque transaction concerne une action)

• ACTION (1) — (0,N) HISTORIQUE_PRIX (Chaque historique de prix concerne une action)

• UTILISATEUR (1) — (0,N) HISTORIQUE_PORTEFEUILLE (Chaque historique de portefeuille concerne un utilisateur)

• UTILISATEUR (1) — (0,N) DIVIDENDE_RECU (Chaque dividende reçu est pour un utilisateur)

• UTILISATEUR (suiveur) (1) — (0,N) SUIVI_JOUEUR

• UTILISATEUR (suivi) (1) — (0,N) SUIVI_JOUEUR

• UTILISATEUR (1) — (0,N) PRET (Chaque prêt est accordé à un utilisateur)

• UTILISATEUR (1) — (0,N) HISTORIQUE_CLASSEMENT_JOUEUR (Chaque entrée de classement concerne un joueur)

Schéma Relationnel de la Base de Données

1. UTILISATEUR (id, email, mot_de_passe, date_inscription, liquidites, derniere_connexion)

• Clé primaire : id

• Unique : email

2. ACTION (id, code, nom, description, prix_actuel, evolution_precedente, date_dividende, montant_dividende)

• Clé primaire : id

• Unique : code

3. POSSESSION_ACTION (id, utilisateur_id, action_id, quantite, prix_moyen_achat)

• Clé primaire : id

• Clés étrangères : utilisateur_id référence UTILISATEUR(id), action_id référence ACTION(id)

• Unique : (utilisateur_id, action_id)

4. TRANSACTION (id, utilisateur_id, action_id, type, quantite, prix_unitaire, montant_total, date_transaction)

• Clé primaire : id

• Clés étrangères : utilisateur_id référence UTILISATEUR(id), action_id référence ACTION(id)

5. HISTORIQUE_PRIX (id, action_id, prix, date)

• Clé primaire : id

• Clé étrangère : action_id référence ACTION(id)

• Unique : (action_id, date)

6. HISTORIQUE_PORTEFEUILLE (id, utilisateur_id, valeur, date)

• Clé primaire : id

• Clé étrangère : utilisateur_id référence UTILISATEUR(id)

• Unique : (utilisateur_id, date)

7. DIVIDENDE_RECU (utilisateur_id, date, montant)

• Clé primaire : (utilisateur_id, date)

• Clé étrangère : utilisateur_id référence UTILISATEUR(id)

8. SUIVI_JOUEUR (id_suivi_joueur, id_suiveur, id_suivi, date_debut_suivi)

• Clé primaire : id_suivi_joueur

• Clés étrangères : id_suiveur référence UTILISATEUR(id),

id_suivi référence UTILISATEUR(id)

• Unique : (id_suiveur, id_suivi)

9. PRETS (id_pret, id_utilisateur, montant_emprunte, taux_interet_annuel,

date_emprunt, date_echeance, montant_rembourse, statut_pret)

• Clé primaire : id_pret

• Clé étrangère : id_utilisateur référence UTILISATEUR(id)

10. HISTORIQUE_CLASSEMENT_JOUEUR (id_historique, id_joueur, date_snapshot, rang, valeur_portefeuille, performance_mensuelle)

• Clé primaire : id_historique

• Clé étrangère : id_joueur référence UTILISATEUR(id)

• Unique : (id_joueur, date_snapshot)

Structure générale du projet

Le projet Virtual Trader est développé en PHP et s'appuie sur une base de données MySQL. L'organisation des fichiers a été pensée pour séparer les différentes préoccupations : la configuration de l'application, la logique (fonctions et traitements), la présentation (templates HTML et CSS), et les scripts accessibles publiquement par l'utilisateur. Cette structure modulaire vise à faciliter la compréhension, la maintenance et l'évolution future du projet.

Organisation des dossiers et fichiers

Dossier racine du projet

• auto_simulate_windows_fix.php: Ce script PHP est crucial pour le fonctionnement dynamique du jeu en pseudo temps-réel. Il est conçu pour être exécuté en ligne de commande, par exemple via une tâche planifiée (Windows avec XAMPP). Son rôle est de simuler le passage des tours de jeu : il incrémente la date du jeu (par exemple, d'un mois à chaque exécution), met à jour les prix des actions de manière aléatoire (en respectant les contraintes de variation et le prix plancher), et distribue les dividendes aux actionnaires concernés. Les instructions spécifiques pour son exécution dans un environnement XAMPP sont détaillées dans le fichier README.md.

• schema_updates.sql: Ce fichier SQL n'est pas directement utilisé par l'application en fonctionnement normal mais sert d'archive pour les modifications apportées à la structure de la base de données durant les phases de développement et de

maintenance. Il peut contenir des instructions ALTER TABLE, CREATE TABLE pour de nouvelles fonctionnalités, ou des insertions de données initiales.

Dossier config

• database.php: Fichier de configuration central et indispensable. Il établit la connexion à la base de données MySQL en utilisant PDO. Il définit les paramètres de connexion tels que l'hôte du serveur de base de données, le nom de la base de données, l'utilisateur et le mot de passe. Ce fichier est inclus au début de la plupart des scripts PHP qui nécessitent une interaction avec la base de données, assurant ainsi une gestion centralisée et sécurisée des informations de connexion.

Dossier includes

• functions.php: Ce fichier regroupe un ensemble de fonctions PHP utilitaires qui sont réutilisées à travers plusieurs parties de l'application. L'objectif est de mutualiser le code et d'éviter la duplication. On y trouve des fonctions pour la gestion des sessions utilisateur la validation et la sécurisation des données entrantes, le formatage de dates ou de nombres, ou toute autre logique métier transversale.

Dossier public

Ce dossier contient les scripts PHP qui génèrent les pages web visibles par les utilisateurs, ainsi que les ressources statiques (CSS, images, JavaScript).

• Scripts PHP (Pages et traitements principaux) :

– index.php: Page d'accueil du site. Elle redirige généralement vers login.php si l'utilisateur n'est pas connecté, ou vers dashboard.php s'il est déjà authentifié.

– register.php: Gère le processus d'inscription des nouveaux utilisateurs. Il affiche le formulaire d'inscription et traite les données soumises pour créer un nouveau compte utilisateur dans la base de données.

– login.php: Gère le processus de connexion des utilisateurs. Il affiche le formulaire de connexion, vérifie les identifiants soumis contre la base de données et initialise la session utilisateur en cas de succès.

– logout.php: Permet à un utilisateur connecté de se déconnecter, en détruisant sa session.

– dashboard.php: Le tableau de bord principal de l'utilisateur une fois connecté. Il offre une vue d'ensemble de son statut dans le jeu : liquidités, valeur totale du portefeuille, liste des actions détenues, et potentiellement un graphique de l'évolution de la valeur du porte-feuille.

– buy_sell.php: Interface permettant aux utilisateurs d'acheter de nouvelles actions ou de vendre celles qu'ils possèdent. Elle interagit avec la base de données pour mettre à jour les possessions d'actions et les liquidités de l'utilisateur, et enregistre chaque transaction.

– classement.php: Affiche le classement général des joueurs, ordonné selon la valeur totale de leur portefeuille. Permet aux joueurs de se comparer.

– dividendes.php: Page dédiée à l'affichage des informations sur les dividendes, qu'il s'agisse des dividendes potentiels des actions du marché ou de l'historique des dividendes perçus par l'utilisateur.

– emprunt.php: Gère la fonctionnalité d'emprunt, permettant aux utilisateurs de contracter des prêts pour augmenter leur capital d'investissement, sous certaines conditions et avec des taux d'intérêt.

– historique_portefeuille.php: Affiche l'historique détaillé de l'évolution de la valeur du portefeuille de l'utilisateur, souvent présenté sous forme de graphique pour une meilleure visualisation des performances passées.

– modifier_mdp.php: Permet à un utilisateur authentifié de changer son mot de passe, en respectant les pratiques de sécurité (demande de l'ancien mot de passe, confirmation du nouveau).

– prix_actions.php: Affiche des informations détaillées sur les actions disponibles, notamment l'historique de l'évolution de leur prix, souvent sous forme de graphiques pour aider à la décision d'investissement.

– rechercher_actions.php: Fournit une fonctionnalité de recherche et de filtrage des actions disponibles sur le marché, selon divers critères (nom, prix, secteur, performance, etc.).

– suivis.php: Permet à un utilisateur de voir la liste des autres joueurs qu'il suit, et potentiellement d'accéder à certaines informations publiques sur leur activité ou leur portefeuille (dans le respect de la confidentialité définie).

– gerer_suivi.php: Script de traitement qui gère les actions de suivre un nouveau joueur ou de ne plus suivre un joueur déjà suivi.

– simuler_tour.php: Interface web, destinée à des fins de test ou d'administration, permettant de déclencher manuellement la simulation d'un tour de jeu (mise à jour des prix, etc.), offrant une alternative au script auto_simulate_windows_fix.php.

– reset_complet_avec_utilisateurs.php: Fonctionnalité d'administration critique, accessible uniquement aux utilisateurs ayant des droits d'administrateur (identifiés par la présence de "admin" dans leur email). Ce script permet de réinitialiser l'ensemble des données du jeu (transactions, possessions, liquidités, etc.). Cette fonction est utile pour redémarrer une session de jeu ou pour des opérations de maintenance majeures.

– ajouter_dividende_test.php: Script de test permettant d'ajouter manuellement des informations de dividendes pour des actions spécifiques, utile durant le développement ou pour corriger des données.

Sous-dossier css

– auth-styles.css: Contient les règles CSS spécifiques à la mise en forme des pages d'authentification (inscription, connexion), pour leur donner une apparence distincte si nécessaire.

Sous-dossier images

– image1.jpg, spongebob_icon.png: Fichiers images utilisés pour l'interface utilisateur du site, comme des illustrations, des icônes (par exemple, l'icône de Bob l'éponge pour le menu utilisateur).

Dossier style

• style.css: Feuille de style CSS principale de l'application. Elle définit l'apparence globale du site : la mise en page des éléments communs (en-tête, pied de page, conteneurs principaux), la typographie, les couleurs, et les styles par défaut des composants HTML.

Dossier templates

header.php: Fichier PHP inclus au début de la plupart des pages web. Il contient la structure HTML commune de l'en-tête : la déclaration DOCTYPE, la balise <head> (avec les métadonnées, le titre de la page, les liens vers les fichiers CSS et potentiellement JavaScript), ainsi que la barre de navigation principale et le menu utilisateur (qui inclut les liens vers le tableau de bord, l'achat/vente, le classement, et le menu déroulant personnel avec les options comme "Joueurs Suivis", "Emprunter", "Historique du portefeuille", "Modifier mot de passe" et "Se déconnecter").

footer.php: Fichier PHP inclus à la fin de la plupart des pages web. Il contient la structure HTML commune du pied de page, comme les informations de copyright, des liens de navigation secondaires, ou des scripts JavaScript à charger en fin de page. Il assure également la fermeture correcte des balises HTML ouvertes dans le header.php.

Objectifs du projet et travail réalisé

Nous avons conçu l'application pour offrir une expérience utilisateur complète, depuis l'inscription jusqu'à la gestion active d'un portefeuille d'actions, en passant par des fonctionnalités sociales et une simulation dynamique du marché.

Gestion des utilisateurs et authentification

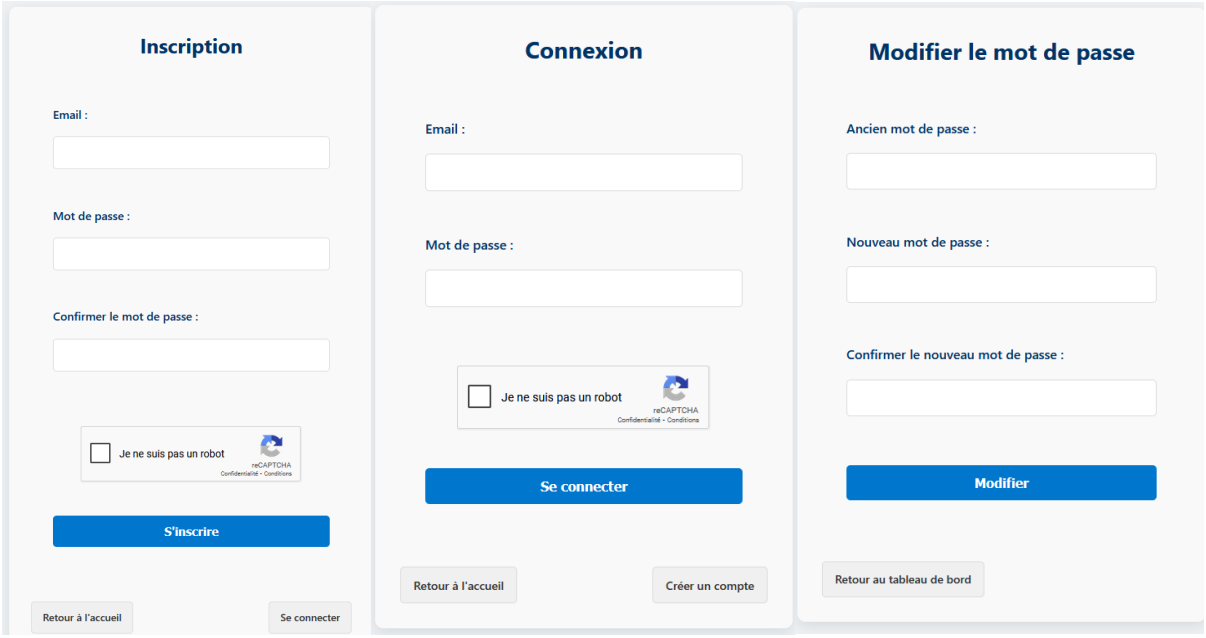
• Objectif : Permettre une gestion sécurisée des comptes utilisateurs.

– Inscription (register.php) : Les nouveaux utilisateurs peuvent créer un compte via un formulaire demandant une adresse e-mail et un mot de passe. Pour la sécurité, les mots de passe sont hachés avant d'être stockés en base de données (table UTILISATEUR).

– Connexion (login.php) : Les utilisateurs existants peuvent se connecter en utilisant leurs identifiants. Une session PHP est initiée pour maintenir l'état de connexion à travers les différentes pages.

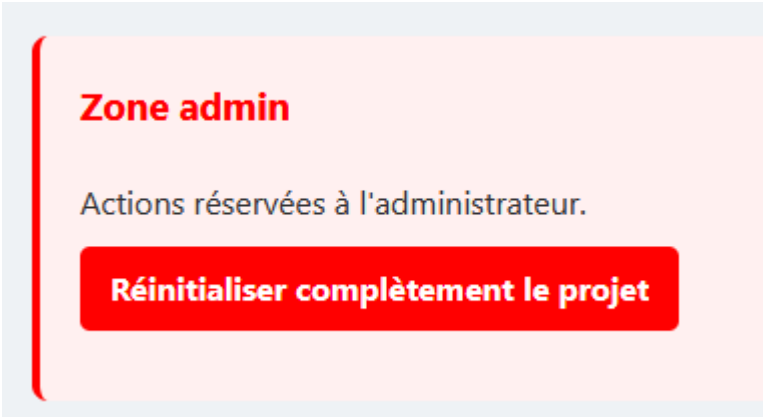
– Déconnexion (logout.php) : Une fonctionnalité de déconnexion sécurisée est disponible, détruisant la session de l'utilisateur.

– Modification du mot de passe (modifier_mdp.php) : Les utilisateurs connectés ont la possibilité de changer leur mot de passe, renforçant la sécurité de leur compte.



– Comptes Administrateur : Une fonctionnalité spécifique a été implémentée : si une adresse e-mail contient le mot "admin" lors de l'inscription, l'utilisateur se voit attribuer des droits d'administration. Ces droits permettent notamment d'accéder à des fonctionnalités de gestion avancée du jeu, comme la réinitialisation complète des données via reset_complet_avec_utilisateurs.php.

• Conclusion : Le système d'authentification et de gestion des utilisateurs est fonctionnel et intègre les mécanismes de base pour la sécurité des comptes.



Fonctionnalités principales du jeu de trading

• Objectif : Simuler les opérations de base d’un marché boursier et la gestion d’un portefeuille.

– Tableau de bord (dashboard.php) : Après connexion, l’utilisateur accède à un tableau de bord personnalisé qui récapitule ses liquidités, la valeur actuelle de ses actions possédées, et la valeur totale de son portefeuille. Un historique de la valeur du portefeuille sur les 12 derniers mois est également affiché (via historique_portefeuille.php et la table HISTORIQUE_PORTEFEUILLE), permettant de suivre ses performances.

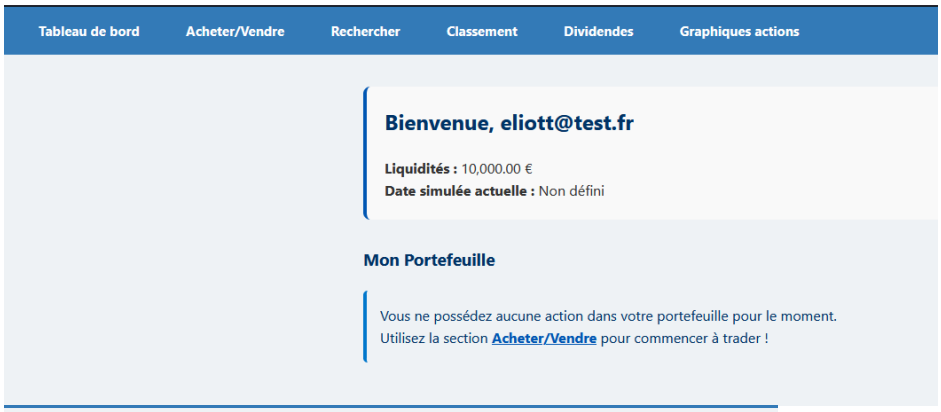
– Achat et Vente d’actions (buy_sell.php, rechercher_actions.php) : Les utilisateurs peuvent consulter la liste des actions disponibles (table ACTION), leur prix actuel, et effectuer des opérations d’achat ou de vente. Les transactions sont enregistrées dans la table TRANSACTION, et les possessions d’actions mises à jour dans POSSESSION_ACTION. Les liquidités sont ajustées en conséquence. La recherche et le filtrage des actions (par nom, prix, etc.) sont possibles, conformément aux attentes.

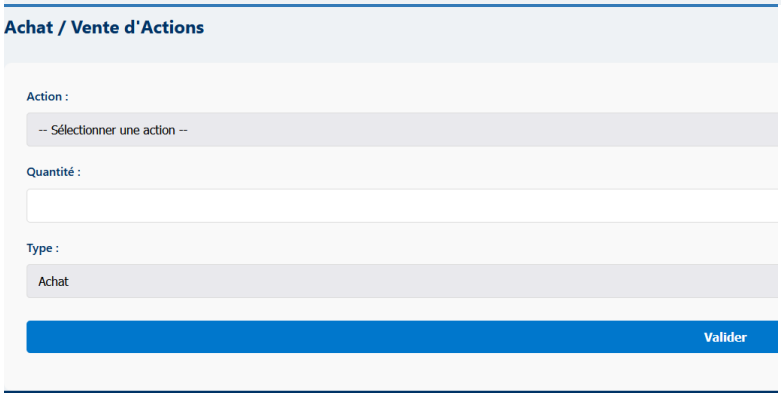
– Visualisation des prix et de leur évolution (prix_actions.php) : L’évolution du prix de chaque action sur les 12 derniers mois est consultable (table HISTORIQUE_PRIX), souvent présentée sous forme de graphique pour une meilleure analyse.

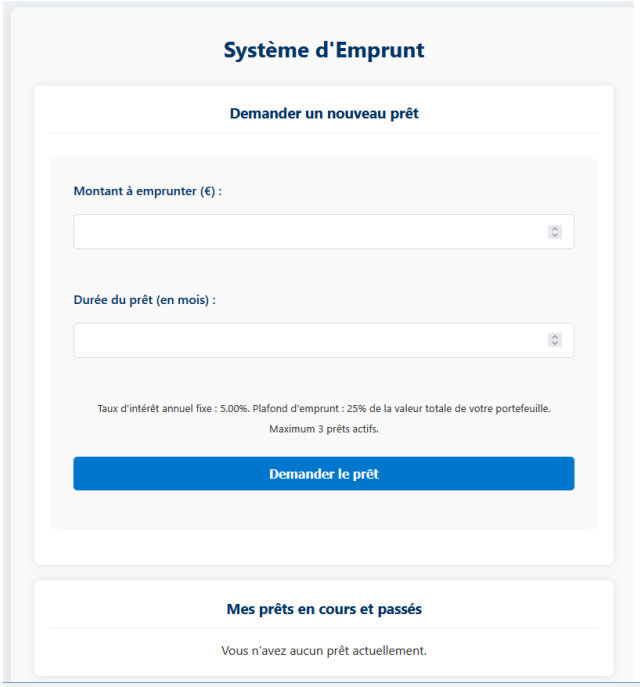
– Dividendes (dividendes.php) : Le système gère la distribution de dividendes. Si une action détenue par un utilisateur arrive à sa date de distribution de dividende (attributs date_dividende et montant_dividende dans la table ACTION), le montant correspondant est crédité sur ses liquidités et enregistré dans DIVIDENDE_RECU.

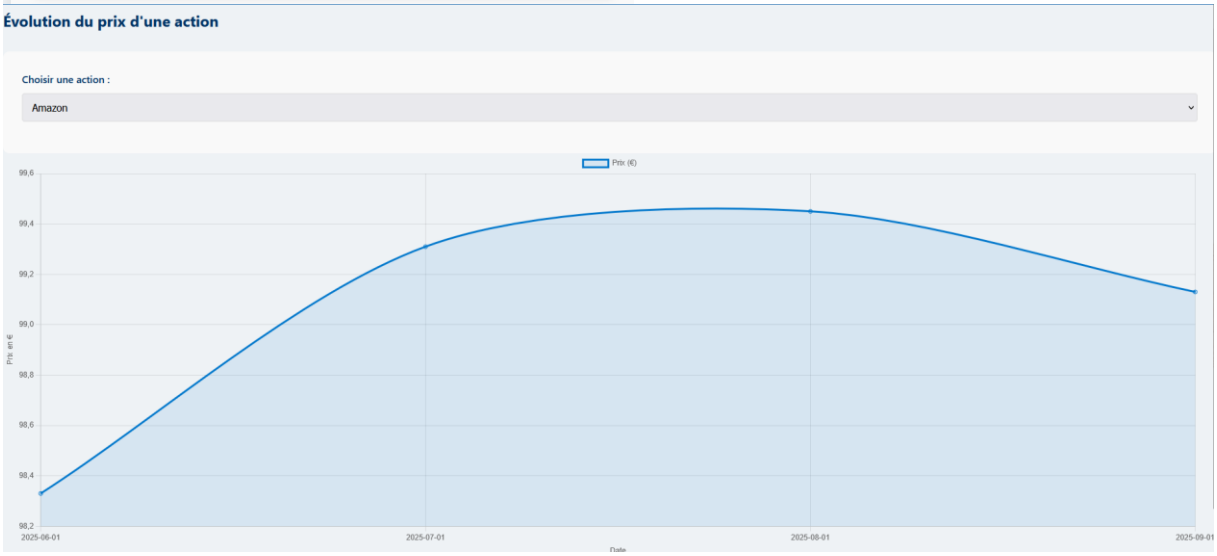
– Emprunts (emprunt.php) : Pour dynamiser le jeu, les utilisateurs peuvent contracter des emprunts (table PRETS) afin d’augmenter leur capital d’investissement. Ces emprunts sont assortis de taux d’intérêt et de dates d’échéance.

• Conclusion : Les fonctionnalités de base du trading sont bien implémentées, offrant une simulation réaliste des opérations boursières. La prise en compte des dividendes et des emprunts enrichit l’expérience de jeu.


Tableau de bord
Acheter/Vendre
Rechercher
Classement
Dividendes
Graphiques actions
Bienvenue, eliott@test.fr
Liquidités : 10,000.00 €
Date simulée actuelle : Non défini
Mon Portefeuille
Vous ne possédez aucune action dans votre portefeuille pour le moment.
Utilisez la section Acheter/Vendre pour commencer à trader !


Achat / Vente d'Actions
Action :
-- Sélectionner une action --
Quantité :
Type :
Achat
Valider





Simulation du marché et dynamique du jeu

• Objectif : Créer un environnement de jeu dynamique avec une évolution des prix et des événements temporels.

– Simulation du temps (auto_simulate_windows_fix.php, simuler_tour.php) : Le jeu progresse par tours, simulant le passage du temps (par exemple, un mois par tour). Le script auto_simulate_windows_fix.php est conçu pour une exécution automatisée (par exemple, toutes les minutes via une tâche planifiée).

À chaque tour :

∗ La date interne du jeu avance.

∗ Les prix des actions sont mis à jour aléatoirement, en respectant une volatilité encadrée (évolution basée sur le mois précédent +/- 3 points, bornée entre -10% et +10% de variation, et un prix plancher de 1€ par action).

– Instructions pour le changement d'heure automatique :

Pour mettre en place cette simulation sur un environnement local XAMPP, l'utilisateur doit ouvrir le shell XAMPP, naviguer vers le dossier du projet(cd C:\xampp\htdocs\virtual_trader-test) et lancer le script avec php auto_simulate_windows_fix.php (pour une simulation où chaque minute réelle correspond à un mois dans le jeu, avec une durée de 60 secondes par exécution) ou php auto_simulate_windows_fix.php 300 (pour une durée de 5 minutes par exécution). Ces informations seront également présentes dans le fichier README.md.

• Conclusion : Le mécanisme de simulation du temps et de mise à jour du marché constitue un élément clé de l'expérience de jeu, rendant le marché dynamique et imprévisible.

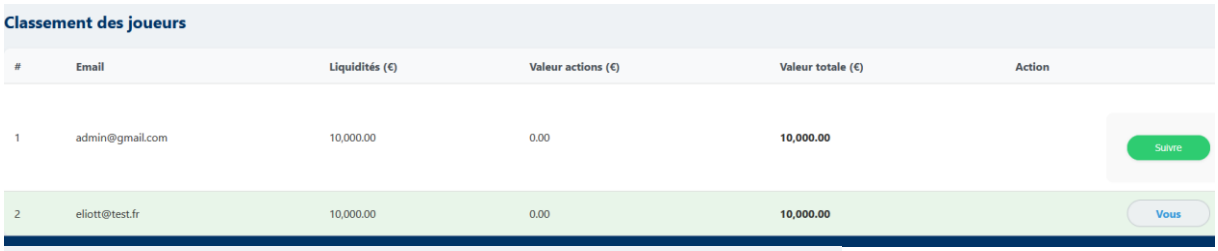
Fonctionnalités sociales et compétitives

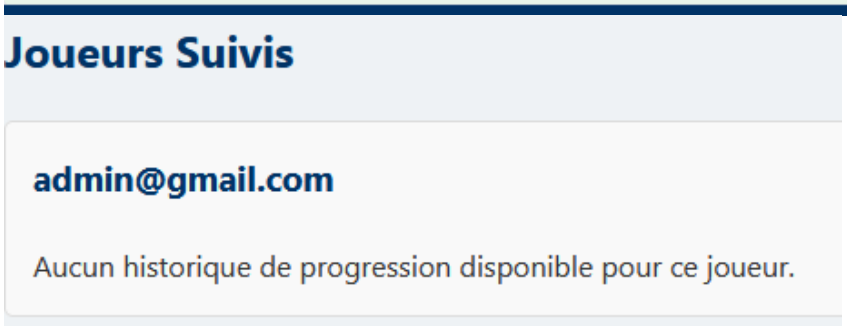
• Objectif : Ajouter une dimension sociale et compétitive au jeu.

– Classement des joueurs (classement.php) : Un classement général des joueurs est disponible, basé sur la valeur totale de leur portefeuille (liquidités + valeur des actions). Cela permet aux joueurs de se comparer et de viser les premières places. Un historique du classement peut être maintenu via la table HISTORIQUE_CLASSEMENT_JOUEUR.

– Suivi de joueurs (gerer_suivi.php, suivis.php) : Les utilisateurs peuvent rechercher d'autres joueurs et choisir de les "suivre". Cela leur permet de consulter une liste des joueurs qu'ils suivent (table SUIVI_JOUEUR) et, de voir certaines de leurs activités ou performances (selon les détails d'implémentation de l'affichage).

• Conclusion : Les aspects compétitifs et sociaux, sont présents et ajoutent de l'intérêt au jeu.





Ergonomie et qualité de l'application

• Objectif : Fournir une application utilisable, avec une interface claire et une bonne expérience utilisateur.

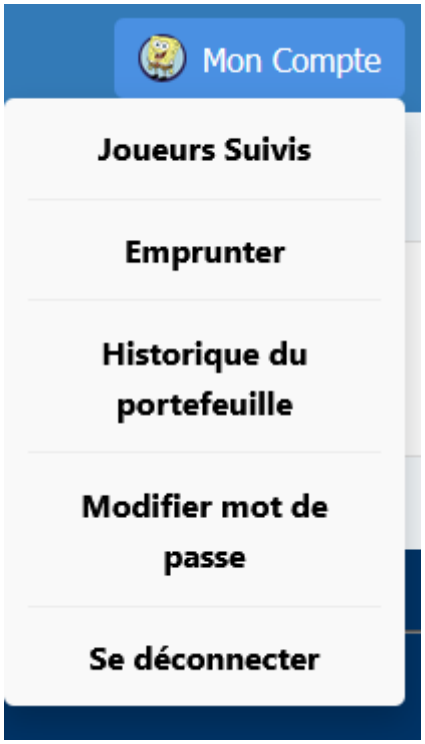
– Navigation : Le site est structuré avec un en-tête (header.php) et un pied de page (footer.php) communs, assurant une navigation. Le menu principal et le menu utilisateur (avec son menu déroulant pour les options personnelles) facilitent l'accès aux différentes sections.

– Interface : L'interface utilisateur est soignée pour être claire et intuitive. Les informations sont présentées de manière organisée, notamment sous forme de tableaux pour les listes d'actions, les possessions, le classement, etc. Des graphiques sont utilisés pour visualiser l'évolution des prix et des portefeuilles.

– Retours utilisateur : Des messages de succès ou d'erreur sont affichés pour informer l'utilisateur du résultat de ses actions (par exemple, après un achat, une vente, ou une modification de mot de passe).

– Contrôle des saisies : Les formulaires intègrent des contrôles de saisie (côté client et/ou serveur) pour s'assurer de la validité des données entrées par les utilisateurs, conformément aux recommandations.

• Conclusion : L'application présente une qualité de finition, avec une ergonomie pensée pour faciliter la prise en main et l'utilisation par les joueurs.



Bilan des fonctionnalités

Le projet Virtual Trader implémente avec succès l'ensemble des fonctionnalités de base demandées dans le cahier des charges, ainsi que certaines améliorations comme la gestion des emprunts et un système d'administration basique. L'architecture de la base de données est robuste et normalisée, et la structure du projet est modulaire. L'application offre une simulation de trading fonctionnelle et engageante, répondant ainsi aux objectifs initiaux du projet.

Le développement du projet Virtual Trader a permis de mettre en application des compétences en conception et en développement web, allant de la modélisation de bases de données à l'implémentation de fonctionnalités interactives et à la gestion de la dynamique d'un jeu de simulation. Nous avons collaboré pour produire une application qui répond aux exigences initiales du cahier des charges tout en intégrant des améliorations et des fonctionnalités spécifiques qui enrichissent l'expérience utilisateur.

L'architecture de la base de données, avec ses schémas Entité-Association et relationnel définis, constitue une fondation solide pour l'application, assurant l'intégrité et la cohérence des données. La structure modulaire du projet, séparant la configuration, la logique métier et la présentation, a facilité le développement et favorise la maintenabilité future de l'application.

Les fonctionnalités implémentées couvrent l'ensemble du cycle de vie d'un utilisateur sur une plateforme de trading simulée : inscription, gestion de portefeuille, achat et vente d'actions, suivi des performances, interaction avec une simulation de marché dynamique, et aspects sociaux tels que le classement et le suivi d'autres joueurs. La prise en compte de spécificités telles que les comptes administrateurs et la simulation automatisée du temps de jeu démontre une compréhension approfondie des enjeux d'une telle application.

Au cours de ce projet, des défis techniques ont été relevés, notamment en ce qui concerne la synchronisation des événements du jeu et la gestion des interactions complexes entre les différentes entités de la base de données. Nous avons décidé de mettre un accent sur l'ergonomie et la clarté de l'interface visant à rendre l'application accessible et agréable à utiliser, même pour des utilisateurs novices en matière de trading.

En conclusion, le projet Virtual Trader représente une réalisation concrète et fonctionnelle, qui non seulement a mis en exécution nos compétences acquises pendant a période d'IF3A mais aussi qui offre également une plateforme de simulation de trading complète et engageante.

Les connaissances acquises lors de ce développement seront précieuses pour de futurs projets, et l'application elle-même pourrait servir de base à des évolutions et des enrichissements ultérieurs

Henni Bilel, Fabrizio Elio, Le Eliott